第1章: 開発環境の構築

> この章では、スマホアプリを作るために必要な「道具」をあなたのPCにそろえます。料理を始める前にキッチンと調理器具を準備するのと同じです。**完全初心者の方でも迷わないよう、初めて出てくるコマンドはすべて1行ずつ解説**します。

---

1. この章でやること（全体像）

これからインストール・設定するものを先に一覧で示します。「何のために入れるのか」を意識すると理解が深まります。

#	入れるもの	役割（なぜ必要か）
1	**Node.js**	TypeScript/JavaScriptをPC上で動かす土台。Expoや各種ツールの動作に必須
2	**VS Code**	コードを書くためのエディタ（高機能メモ帳）
3	**VS Code拡張機能**	コードの色分けや補完を強化し、ミスを減らす
4	**Git**	コードの変更履歴を記録する。後でGitHubやデプロイに使う
5	**Expo関連**	React Nativeアプリを作る本命の道具一式
6	**スマホ側のExpo Goアプリ**	作りかけアプリを自分のスマホで即確認する

> **作業の進め方:** 上から順番に進めれば必ず動く状態になります。途中でエラーが出ても焦らないでください。よくあるエラーと対処法は各ステップの最後にまとめています。

---

2. Node.js のインストール

2.1 Node.js とは

Node.js（ノードジェイエス） は、本来ブラウザの中でしか動かない JavaScript を、**あなたのPCの上で直接動かせるようにする実行環境**です。Expoや開発ツールはすべてこのNode.jsの上で動くため、最初に入れる必要があります。

> **「ランタイム（runtime）」とは？**「プログラムを実行する環境」のこと。Node.jsはJavaScript/TypeScriptのランタイムです。キッチン（Node.js）が無いと料理（プログラム）が作れない、とイメージしてください。

2.2 インストール手順

1. ブラウザで公式サイト **https://nodejs.org/** を開きます。
2. 「**LTS**」と書かれたボタンをクリックしてインストーラをダウンロードします。
3. ダウンロードしたファイルを実行し、表示される画面で基本的に「Next（次へ）」を押し続けてインストールします。

> 「**LTS**」とは？ Long Term Support（ロング・ターム・サポート＝長期サポート版）の略。「安定していて長く使える推奨版」という意味です。新しい機能満載の最新版より、初心者はLTSを選ぶのが安全です。本書執筆時点では Node.js 20.x 以上を推奨します。

2.3 インストールできたか確認する

インストールが終わったら、**ターミナル**を開いて確認します。

> **ターミナルの開き方:** - **Windows**: スタートメニューで「PowerShell」と検索して起動。 - **Mac**: 「Launchpad」→「その他」→「ターミナル」、または Command+Space で「ターミナル」と検索。

ターミナルに次のコマンドを打って Enter キーを押します。

コピー

```
node -v
# node : Node.js本体を呼び出すコマンド
# -v   : version（バージョン）を表示するオプション。「今入っているNode.jsの版を教えて」という意味
# 実行すると v20.x.x のようにバージョン番号が表示されれば成功
```

コピー

```
npm -v
# npm  : Node.jsに付属するパッケージマネージャ（部品管理ツール）のコマンド
# -v   : こちらもバージョン表示。Node.jsを入れるとnpmも自動で一緒に入る
# 10.x.x のような番号が出れば成功
```

▼ 実行結果の例:

コピー

```
v20.11.1
10.2.4
```

> **`v20.11.1` のような番号が出れば成功です。**「'node' は認識されていません」のようなエラーが出る場合は、Node.jsのインストールに失敗しているか、ターミナルの再起動が必要です。一度ターミナルを閉じてもう一度開き直してから、再度 `node -v` を試してください。
>
> **「npm（エヌピーエム）」とは？** Node Package Manager の略。「パッケージ（他人が作った便利なプログラム部品）」をインターネットからダウンロードして管理するツールです。アプリ作りでは大量の部品を使うので、これが活躍します。

---

3. VS Code のインストールと設定

3.1 VS Code とは

VS Code（ブイエス・コード、正式名称 Visual Studio Code） は、Microsoft社が無料で提供している**コードエディタ**（コードを書くための高機能なメモ帳）です。世界で最も使われているエディタの一つで、便利な拡張機能が豊富です。

3.2 インストール手順

1. ブラウザで **https://code.visualstudio.com/** を開きます。
2. 自分のOS（Windows / Mac）用のダウンロードボタンを押します。
3. ダウンロードしたファイルを実行してインストールします。

> **Windowsの方へ（おすすめ設定）**：インストール中に「PATH への追加」「エクスプローラーのコンテキストメニューに『Codeで開く』を追加」といったチェック項目が出たら、全部チェックを入れておくと後で便利です。

3.3 おすすめ拡張機能を入れる

VS Code の左端にある四角が並んだアイコン（**拡張機能ビュー**）をクリックし、検索欄に名前を入れて「Install（インストール）」を押します。

拡張機能名	役割
ESLint	コードの間違いやクセを自動でチェックして教えてくれる
Prettier - Code formatter	コードの見た目（インデントや改行）を自動で整える
ES7+ React/Redux/React-Native snippets	Reactのコードを短い入力で素早く書ける「定型文」集
Expo Tools	Expo関連ファイルの補完やヘルプを提供
Japanese Language Pack（任意）	VS Codeのメニューを日本語化する

> 「**拡張機能（extension）」とは？** VS Codeに後から機能を追加する"アドオン"のことです。スマホアプリを追加でインストールするのと同じ感覚で、エディタを便利にできます。

---

4. Git のインストール

4.1 Git とは

Git（ギット） は、「いつ・誰が・どのファイルを・どう変更したか」を記録する**バージョン管理ツール**です。作業を間違えても過去の状態に戻せる「タイムマシン付きノート」のようなものです。後の章で GitHub へのアップロードやアプリの公開に使います。

4.2 インストール手順

- **Windows**: **https://git-scm.com/** から「Download for Windows」を選び、インストーラを実行。基本は「Next」を押し続けてOKです。
- **Mac**: ターミナルで次を実行すると、開発ツール一式の案内が出るので従ってインストールします。

コピー

```
git --version
# git       : Gitを呼び出すコマンド
# --version : バージョンを表示するオプション（-v の長い書き方）
# Macでは未インストールの場合、ここで「開発ツールをインストールしますか?」と案内が出る
# git version 2.x.x のように表示されれば成功
```

▼ 実行結果の例:

コピー

```
git version 2.43.0
```

4.3 Gitの初期設定（名前とメールの登録）

Gitに「あなたが誰か」を教えます。これは変更履歴に記録される署名になります。一度だけ設定すればOKです。

コピー

```
git config --global user.name "あなたの名前"
# git config       : Gitの設定を読み書きするコマンド
# --global         : 「このPC全体の共通設定にする」オプション（プロジェクトごとでなく全体）
# user.name        : 設定する項目名。「ユーザー名（署名に使う名前）」
# "あなたの名前"   : 設定する値。ダブルクォートで囲む。日本語でもローマ字でも可
```

コピー

```
git config --global user.email "あなたのメールアドレス"
# user.email : 設定項目「メールアドレス」。GitHubで使う予定のアドレスを推奨
```

> **設定できたか確認:** `git config --global user.name` のように値を省いて実行すると、今設定されている値が表示されます。

---

5. Expo のセットアップ（本書のメイン）

ここからが本命です。Expoを使って実際にアプリのひな形を作り、自分のスマホで動かすところまでやります。

5.1 Expo とは（おさらい）

Expo（エクスポ） は、React Native アプリ開発を劇的に簡単にする道具一式です。プロジェクト作成・実機確認・ビルド・公開までを支援します。第0章の比較表で見た通り、初心者に最適な選択肢です。

> **事前インストールは不要:** Expoの便利なところは、専用ツールを事前にPCへインストールしなくても、`npx`（ご紹介済みの「その場で取ってきて実行するコマンド」）を使えばすぐ始められる点です。

5.2 スマホに Expo Go アプリを入れる

作りかけのアプリを自分のスマホで確認するために、**Expo Go（エクスポ・ゴー）** という無料アプリをスマホに入れます。

- **iPhone**: App Store で「**Expo Go**」を検索してインストール。
- **Android**: Google Play で「**Expo Go**」を検索してインストール。

> **Expo Go の役割:** あなたのPCで動かしている開発中アプリを、このExpo Goアプリが"読み込んで"スマホ画面に表示してくれます。アプリを毎回インストールし直す必要がなく、コードを保存した瞬間にスマホの画面が更新されます（これを**ホットリロード**と呼びます）。

5.3 最初のプロジェクトを作る

ターミナルで、プロジェクトを置きたい場所（例: デスクトップや書類フォルダ）へ移動してから、次のコマンドを実行します。

フォルダ移動の例（必要に応じて）：

コピー

```
cd Desktop
# cd      : change directory（カレントディレクトリ＝今いるフォルダ を変える）コマンド
# Desktop : 移動先のフォルダ名。これでデスクトップの中に入る
```

コピー

```
npx create-expo-app@latest my-books-app
# npx                 : パッケージを一時的に取得して即実行するコマンド
# create-expo-app     : Expoアプリのひな形（テンプレート）を生成する公式ツール
# @latest             : 「最新版のツールを使う」という指定（@ の後ろにバージョンを書ける）
# my-books-app        : これから作るプロジェクト（フォルダ）の名前。好きな名前でOK（半角英数とハイフン推奨）
# 実行すると my-books-app という名前のフォルダが作られ、必要な部品が自動でダウンロードされる
```

実行中に時間がかかります: 大量の部品をインターネットからダウンロードするため、数分かかることがあります。「Installing dependencies...（依存パッケージをインストール中）」と表示されたら、完了まで待ちましょう。

完了したら、作られたフォルダの中へ移動します。

コピー

```
cd my-books-app
# cd          : フォルダを移動するコマンド
# my-books-app : さっき作ったプロジェクトフォルダ名。この中で以降の作業をする
```

5.4 開発サーバーを起動して実機で確認する

いよいよアプリを起動します。

コピー

```
npx expo start
# npx expo    : Expoのコマンドラインツールを実行
# start       : 「開発サーバーを起動する」サブコマンド
# 実行すると、ターミナルに大きなQRコードと操作メニューが表示される
```

▼ 実行結果の例（一部）：

コピー

```
› Metro waiting on exp://192.168.0.5:8081
› Scan the QR code above with Expo Go (Android) or the Camera app (iOS)

› Press a │ open Android
› Press i │ open iOS simulator
› Press w │ open web
```

> 「**Metro（メトロ）**」とは？ React Nativeのコードを、スマホが理解できる形にリアルタイムで変換する仕組み（バンドラー）の名前です。`npx expo start` を実行するとこのMetroが裏で起動し、コードを保存するたびに自動でアプリへ反映してくれます。

スマホで確認する方法:

- **Android**: Expo Goアプリを開き、「Scan QR code」でターミナルのQRコードを読み取る。
- **iPhone**: 標準の「カメラ」アプリでQRコードを読み取り、表示される通知（バナー）をタップ。

> **重要な条件**: PCとスマホが**同じWi-Fi（ネットワーク）に接続**されている必要があります。会社や学校のネットワークだと通信が遮断されてうまくいかないことがあります。その場合は `npx expo start --tunnel`（トンネルモード。インターネット経由で接続する方法）を試してください。

スマホ画面に初期画面が表示されたら成功です！ 🎉 試しに、プロジェクト内の `app/(tabs)/index.tsx` というファイルの文字を書き換えて保存すると、スマホ画面が即座に変わるのを確認できます。

> **ターミナルを止めるには:** `Ctrl + C`（WindowsもMacも同じ）を押すと開発サーバーが停止します。`Ctrl` は「コントロールキー」、`C` を同時に押します。

5.5 PCのエミュレータ/シミュレータで確認したい場合（任意）

スマホを使わずPC内で確認したい場合は、以下を用意します。**必須ではありません**（実機があれば不要）。

- **Android エミュレータ（Windows/Mac両方）**: **Android Studio** をインストールし、その中の「Device Manager」で仮想スマホを作成。`npx expo start` 後にターミナルで `a` キーを押すと起動します。
- **iOS シミュレータ（Macのみ）**: App Storeから **Xcode** をインストール。`npx expo start` 後に `i` キーを押すと起動します。

> **WindowsユーザーはiOSの確認ができないの？** PC内のiOSシミュレータはMac専用です。しかしWindowsでも、**実機のiPhone＋Expo Go**を使えばiOSでの見た目を確認できます。さらに公開用ビルド（第10章のEAS）はクラウドで行うため、**Windowsだけで最後まで完走できます。**

6.【参考】Expoを使わない場合の環境構築（React Native CLI）

> ここは**参考セクション**です。本書はExpoで進めるので、読み飛ばして第2章へ進んでも構いません。ただし「Expoを使わない方法」「Expoとの違い」「後から切り替える方法」を知っておくと、技術選択の視野が広がります。

6.1 React Native CLI とは

React Native CLI（リアクト・ネイティブ・シーエルアイ） は、Expoを使わずに「素のReact Native」を直接扱う公式の道具です。第0章の比較表の通り、自由度が最大の代わりに環境構築の手間が大きいのが特徴です。「**bare workflow（ベア・ワークフロー＝むき出しの構成）**」とも呼ばれます。

6.2 追加で必要になるもの

Expoでは不要だった、以下の本格的なネイティブ開発ツールが必要になります。

OS	必要なもの	役割
iOSアプリを作る	**Mac本体 ＋ Xcode**	AppleのiOS開発環境。**Windowsでは不可**
Androidアプリを作る	**Android Studio** ＋ JDK	Googleの Android 開発環境。OSはWin/Mac両対応
共通	**Watchman**（Mac推奨）	ファイル変更を監視する補助ツール

> 「**JDK**」とは？ Java Development Kit の略。Androidアプリのビルドに必要なJava言語の開発キットです。Android Studioが面倒を見てくれることが多いです。

6.3 環境構築の大まかな流れ

公式ドキュメント（https://reactnative.dev/docs/set-up-your-environment ）の手順に沿いますが、概略は次の通りです。

コピー

```
# (1) 新規プロジェクトを作成する（Expoの create-expo-app に相当）
npx @react-native-community/cli@latest init MyBooksApp
# npx                                  : その場でツールを取得して実行
# @react-native-community/cli@latest   : React Native公式CLIの最新版
# init                                 : 新規プロジェクトを初期化するサブコマンド
# MyBooksApp                           : プロジェクト名（CLIではキャメルケース推奨）
```

コピー

```
# (2) iOS用の追加部品を取り込む（Macのみ。CocoaPodsという仕組みを使う）
cd MyBooksApp/ios && pod install && cd ..
# cd MyBooksApp/ios : iOS設定フォルダへ移動
# pod install       : CocoaPods（iOSの部品管理ツール）で必要部品を取り込む
# && cd ..          : 成功したら一つ上のフォルダへ戻る（&& は「前が成功したら次を実行」）
```

コピー

```
# (3) アプリを起動する
# Androidエミュレータ/実機で起動
npx react-native run-android
# iOSシミュレータで起動（Macのみ）
npx react-native run-ios
# Expoの「QRコードで即確認」とは異なり、エミュレータ/実機へ直接ビルドして転送する
```

> **Expoとの体験の違い:** CLIでは、起動のたびにネイティブのビルド（Xcode/Android Studioによるコンパイル）が走るため**時間がかかり**、最初の環境構築でつまずきやすいです。一方で、特殊なネイティブ機能を自由に組み込めます。

6.4 Expo と CLI の切り替え方

実務でよくあるのが「**最初はExpoで手軽に始め、後から自由度が必要になってCLI構成へ移行する**」というパターンです。Expoはこの移行を公式にサポートしています。

◆ Expo（managed）→ CLI構成（bare）へ移行する

コピー

```
npx expo prebuild
# npx expo  : Expoのコマンドラインツール
# prebuild  : プロジェクトに ios/ と android/ という「ネイティブのフォルダ」を生成する
#             これにより、XcodeやAndroid Studioで直接開いて編集できる「bare構成」になる
```

このコマンドを実行すると、それまで隠されていた `ios/` と `android/` フォルダが出現し、ネイティブコードを直接いじれるようになります。**Expoの便利機能（EASビルド等）はその後も使えます。**これを「**Continuous Native Generation（CNG）**」と呼びます。

> 「**prebuild」のポイント:** `ios/` `android/` フォルダは `app.json`（設定ファイル）の内容から自動生成されます。そのため、誤って消してしまっても `npx expo prebuild --clean` で作り直せます。「設定ファイルが正、ネイティブフォルダは生成物」という考え方です。

◆ CLI構成のプロジェクトでExpoの便利機能だけ使う

逆に、素のCLIで始めたプロジェクトに、後からExpoのライブラリ（`expo-camera` など）やEASビルドを部分的に導入することもできます。現在のExpoは「**全部入りか、ゼロか**」ではなく「**必要なものだけ足す**」設計になっています。

> **結論（初心者への推奨）：まずはExpoで始める**のが圧倒的におすすめです。Expoで作っていて「どうしてもExpoが対応していないネイティブ機能が必要」になった、その時に初めて `npx expo prebuild` でCLI構成へ移行すれば十分です。最初からCLIで苦労する必要はありません。

---

7. プロジェクトのフォルダ構成を眺めてみる

`create-expo-app` で作られたプロジェクトには、たくさんのファイル・フォルダがあります。今すべてを理解する必要はありません。「こういうものがあるんだ」と眺めるだけでOKです。VS Codeでプロジェクトフォルダを開いてみましょう。

コピー

```
my-books-app/
├── app/                  ← 【重要】画面ファイルを置く場所。ここにファイルを作ると画面が増える
│   ├── (tabs)/           ← 下タブで切り替わる画面群（Expo Routerの仕組み）
│   │   ├── index.tsx     ← 最初に表示される画面
│   │   └── explore.tsx   ← 2つ目のタブの画面
│   └── _layout.tsx       ← アプリ全体の画面の枠組み（レイアウト）を定義
├── components/           ← 使い回す部品（コンポーネント）を置く場所
├── constants/            ← 色などの定数（変わらない値）をまとめる場所
├── assets/               ← 画像やフォントなどの素材を置く場所
├── app.json              ← アプリの名前・アイコン・設定を書くファイル
├── package.json          ← 使っている部品の一覧と、実行コマンドの定義
├── tsconfig.json         ← TypeScriptの設定ファイル
└── node_modules/         ← ダウンロードした部品の本体（自動生成。自分では触らない）
```

> **VS Codeでプロジェクトを開くには:** VS Codeのメニュー「ファイル」→「フォルダーを開く」で `my-books-app` フォルダを選びます。または、ターミナルでプロジェクト内にいる状態で `code .` と打つと、その場でVS Codeが開きます（`code` はVS Codeを起動するコマンド、`.` は「今いるフォルダ」の意味）。
>
> **`package.json` の `scripts` とは？** `package.json` の中に `"scripts"` という欄があり、`npm start` などの短いコマンドで何が実行されるかが定義されています。例えば `"start": "expo start"` と書いてあれば、`npm start` と打つだけで `expo start` が動きます。本書では分かりやすさのため `npx expo start` と直接書きますが、`npm start` でも同じです。

---

8. この章のまとめ

- **Node.js**（土台）、**VS Code**（エディタ）、**Git**（履歴管理）をPCに入れた
- **Expo** でプロジェクトを作り、**Expo Go** を使って自分のスマホで動作確認できるようになった
- （**参考**） Expoを使わない **React Native CLI** 構成の存在と、`npx expo prebuild` による**切り替え方**を理解した
- プロジェクトのフォルダ構成をざっくり把握した

> **次の章へ:** 環境が整いました。第2章では、アプリのロジックを書くための言語 **TypeScript** の基礎を学びます。ここで作った `my-books-app` は後の章でも使い続けるので、消さずに残しておいてください。

よくあるトラブルと対処法

症状	対処法
`node` コマンドが認識されない	ターミナルを開き直す。それでも駄目ならNode.jsを再インストール
QRコードを読んでもアプリが表示されない	PCとスマホが**同じWi-Fi**か確認。駄目なら `npx expo start --tunnel`
`npx create-expo-app` が途中で止まる	ネットワーク接続を確認し、再実行。社内ネットの場合は別回線を試す
スマホで「Something went wrong」表示	ターミナルのエラー文を確認。多くはコードの保存ミス。`Ctrl+C` で止めて再起動